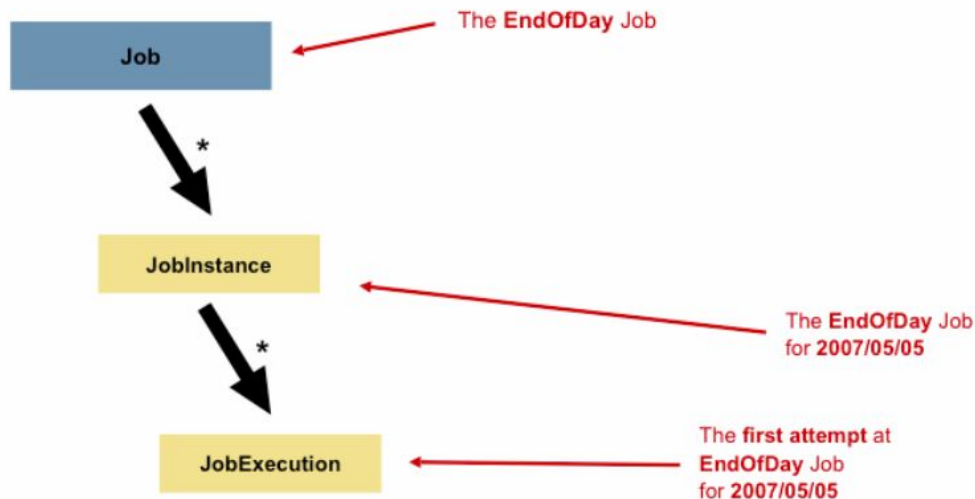


Trabajo

Un trabajo es una entidad que encapsula un proceso por lotes completo y es solo la parte superior de una jerarquía general.



En *Spring Batch*, un trabajo es simplemente un contenedor de pasos. Combina varios pasos que pertenecen lógicamente juntos en un flujo y permite la configuración de propiedades globales para todos los pasos, como la reiniciabilidad. La configuración del trabajo contiene:

- El nombre simple del trabajo.
- Definición y ordenación de Pasos.
- Si el trabajo se puede reiniciar o no.

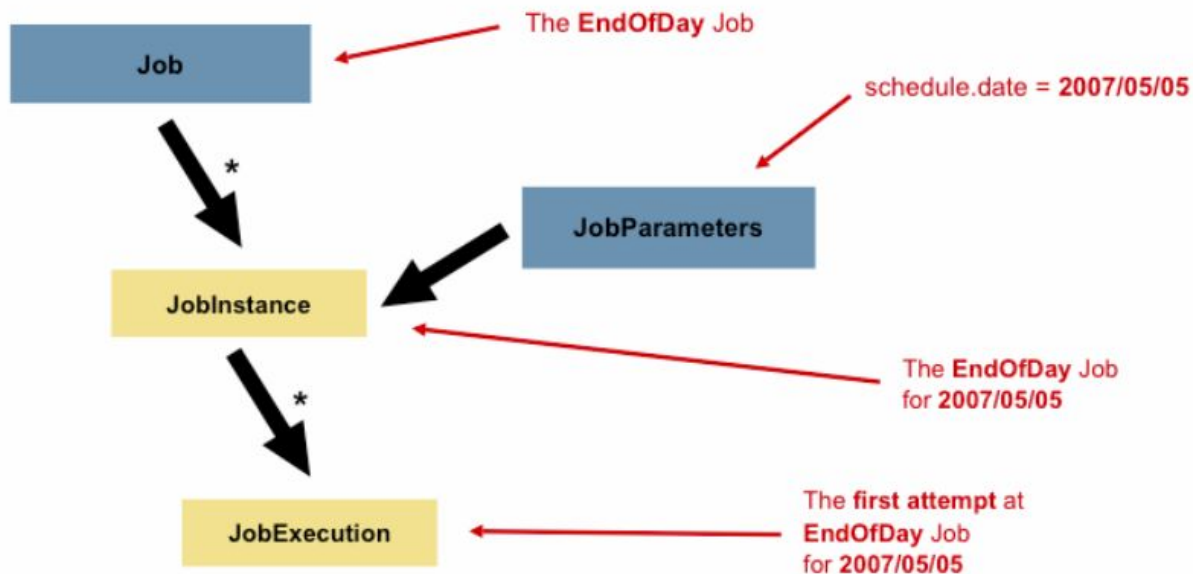
Instancia de trabajo

Una `JobInstance` se refiere al concepto de ejecución de un trabajo lógico. Por lo tanto, cada `JobInstance` puede tener múltiples ejecuciones y solo se puede ejecutar un `JobInstance` correspondiente a un `Job` particular e identificando `JobParameters` en un momento dado.

La definición de `JobInstance` no tiene absolutamente ninguna relación con los datos que se cargarán. Depende completamente de la implementación de `ItemReader` utilizada para determinar cómo se cargarán los datos.

Parámetros del trabajo

JobParameters es un conjunto de parámetros que se utilizan para iniciar un trabajo por lotes. Se pueden utilizar para identificación o incluso como datos de referencia durante la ejecución.



Ejecución del trabajo

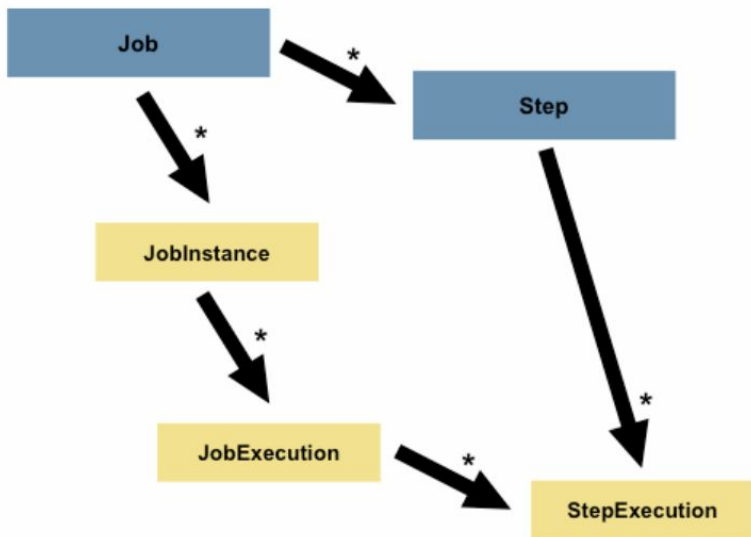
Una `JobExecution` se refiere al concepto técnico de un único intento de ejecutar un `Job`. Una ejecución puede finalizar con éxito o fracaso, pero la `JobInstance` correspondiente a una ejecución determinada no se considerará completa a menos que la ejecución se complete con éxito.

Un trabajo define qué es un trabajo y cómo se ejecutará, y `JobInstance` es un objeto puramente organizativo para agrupar ejecuciones, principalmente para permitir una semántica de reinicio correcta. Sin embargo, una `JobExecution` es el mecanismo de almacenamiento principal de lo que realmente sucedió durante una ejecución y, como tal, contiene muchas más propiedades que deben controlarse y persistir:

Propiedad	Descripción
estado	Un objeto BatchStatus que indica el estado de la ejecución. Mientras se ejecuta, es BatchStatus.STARTED, si falla, es BatchStatus.FAILED, y si finaliza exitosamente, es BatchStatus.COMPLETED
hora de inicio	Un java.util.Date que representa la hora actual del sistema cuando se inició la ejecución.
hora de finalización	Un java.util.Date que representa la hora actual del sistema cuando finalizó la ejecución, independientemente de si fue exitosa o no.
estado de salida	El ExitStatus que indica el resultado de la ejecución. Es muy importante porque contiene un código de salida que se devolverá a la persona que llama.
creartiempo	Un java.util.Date que representa la hora actual del sistema cuando JobExecution persistió por primera vez. Es posible que el trabajo aún no se haya iniciado (y por lo tanto no tenga hora de inicio), pero siempre tendrá un createTime, que es requerido por el marco para administrar ExecutionContexts a nivel de trabajo.
última actualización	Un java.util.Date que representa la última vez que persistió una JobExecution.
contexto de ejecución	La 'bolsa de propiedades' que contiene los datos del usuario que deben conservarse entre ejecuciones.
fracasoExcepciones	La lista de excepciones encontradas durante la ejecución de un trabajo. Estos pueden resultar útiles si se encuentra más de una excepción durante el error de un trabajo.

Step

A Step is a domain object that encapsulates an independent, sequential phase of a batch job. Therefore, every Job is composed entirely of one or more steps. A Step contains all of the information necessary to define and control the actual batch processing. This is a necessarily vague description because the contents of any given Step are at the discretion of the developer writing a Job. A Step can be as simple or complex as the developer desires. A simple Step might load data from a file into the database, requiring little or no code. (depending upon the implementations used) A more complex Step may have complicated business rules that are applied as part of the processing. As with Job, a Step has an individual StepExecution that corresponds with a unique JobExecution.



Step Execution

A StepExecution represents a single attempt to execute a Step. A new StepExecution will be created each time a Step is run, similar to JobExecution. However, if a step fails to execute because the step before it fails, there will be no execution persisted for it. A StepExecution will only be created when its Step is actually started.

The following is a listing of the properties for StepExecution:

Property	Description
status	A BatchStatus object that indicates the status of the execution. While it's running, the status is BatchStatus.STARTED, if it fails, the status is BatchStatus.FAILED, and if it finishes successfully, the status is BatchStatus.COMPLETED
startTime	A java.util.Date representing the current system time when the execution was started.
endTime	A java.util.Date representing the current system time when the execution finished, regardless of whether or not it was successful.
exitStatus	The ExitStatus indicating the result of the execution. It is most important because it contains an exit code that will be returned to the caller. See chapter 5 for more details.
executionContext	The 'property bag' containing any user data that needs to be persisted between executions.
readCount	The number of items that have been successfully read
writeCount	The number of items that have been successfully written
commitCount	The number transactions that have been committed for this execution
rollbackCount	The number of times the business transaction controlled by the Step has been rolled back.
readSkipCount	The number of times read has failed, resulting in a skipped item.
processSkipCount	The number of times process has failed, resulting in a skipped item.
filterCount	The number of items that have been 'filtered' by the ItemProcessor.
writeSkipCount	The number of times write has failed, resulting in a skipped item.

Steps type

A Job will be comprised of one or more steps. The steps can be of two types:

- **Chunk:** It involves reading an input, processing it based on the business logic and then aggregating it until the commit-interval is reached and finally writing out the chunk of data output to a file or database table. Usually used in scenarios where multiple aggregated steps need to be run. It's formed by 3 components:
 - Reader: Read input data of step.
 - Processor: Process the data obtained by the Reader.
 - Writer: Write the processed data by the Processor.

If need to create your own readers or writers, it can be implemented the next interfaces:

- Reader : `ItemReader`.
- Processor: `ItemProcessor`.
- Writer: `ItemWriter`.